

MULTI-AGENT AI DOCUMENT & PPT GENERATOR

Project Documentation

Enterprise-oriented AI system for document analysis, web research, enterprise RAG, and editable DOCX/PPTX generation.

Developed by
Shreyash Musmade

1. Project Overview

Multi-Agent AI Document & PPT Generator is an enterprise-oriented Multi-Agent AI chatbot proof of concept. The system is designed to understand user requests, analyze uploaded documents and presentation templates, perform web research, retrieve enterprise knowledge through Retrieval-Augmented Generation (RAG), and generate editable DOCX documents and PPTX presentations.

The project uses Google Gemini as the primary LLM, Pinecone for vector storage and retrieval, FastAPI for the backend API layer, Streamlit for the user interface, and document-processing libraries for PDF, DOCX, PPTX, image, and OCR workflows.

2. Problem Statement

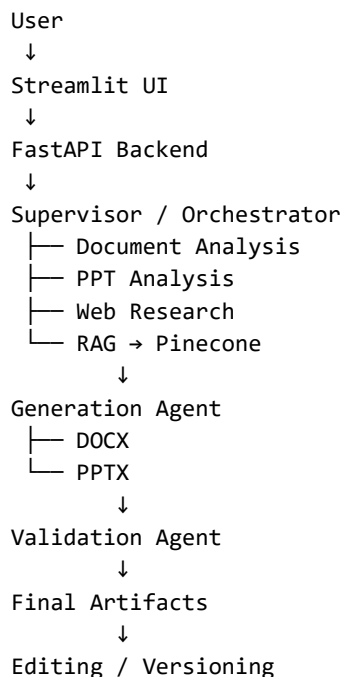
Organizations often spend significant time manually analyzing business documents, understanding existing templates, researching current information, finding enterprise-specific knowledge, and preparing professional documents and presentations. Repeated editing and maintaining formatting further increases the effort.

3. Objectives

The main objectives are to build a specialized multi-agent workflow, analyze PDF/DOCX/PPTX/image inputs, perform web research, implement enterprise RAG with Pinecone, generate editable DOCX/PPTX artifacts, support conversational editing, validate artifacts, and provide source/citation and version-related functionality.

4. Proposed Solution

The application separates responsibilities across specialized agents. A Supervisor/Orchestrator coordinates the workflow and delegates tasks to document analysis, PPT analysis, web research, RAG, generation, editing, and validation components.



5. System Architecture

Frontend

Streamlit provides the interactive user interface for file upload, chat interaction, and artifact workflows.

Backend

FastAPI provides APIs for upload, chat, artifact, version, and health operations.

AI Layer

Google Gemini provides language understanding, analysis, reasoning, and generation.

Knowledge Layer

Pinecone provides vector storage and semantic similarity search for enterprise RAG.

Processing Layer

PyMuPDF, python-docx, python-pptx, Pillow, and Tesseract/pytesseract support document, presentation, image, and OCR processing.

Background Processing

Celery and Redis are included for asynchronous/background-processing support.

6. Multi-Agent Architecture

Agent	Responsibility	Output
Supervisor Agent	Coordinates the complete workflow	Agent routing/orchestration
Document Analysis Agent	Analyzes uploaded documents	Content and structure
PPT Analysis Agent	Analyzes presentations/templates	Slide/layout information
Web Research Agent	Retrieves current external	Research and sources

	information	
RAG Agent	Retrieves enterprise knowledge	Relevant context
Generation Agent	Creates requested artifacts	DOCX/PPTX
Editing Agent	Processes natural-language edits	Updated artifacts
Validation Agent	Validates generated artifacts	Validation results

7. Agent Responsibilities

Supervisor / Orchestrator

Understands the user request, determines the workflow, coordinates agents, and combines results.

Document Analysis

Processes PDF, DOCX and image-related content and extracts useful information.

PPT Analysis

Examines slide structure, titles, text, layouts, existing content, and presentation style.

Web Research

Retrieves external information for current-knowledge requests and supports source tracking.

RAG

Retrieves relevant enterprise knowledge from the vector database and supplies contextual information.

Generation

Produces structured document and presentation content.

Editing

Provides conversational modification of generated artifacts.

Validation

Performs validation before final artifact delivery.

8. Enterprise RAG Pipeline

Retrieval-Augmented Generation combines retrieved enterprise context with the language model so that generated responses can use organization-specific information.

```

Enterprise Documents
↓
Text Extraction
↓
Chunking
↓
Gemini Embeddings
↓
Pinecone Vector Database
↓
User Query
↓
Similarity Search

```

↓
Relevant Context
↓
Google Gemini
↓
Grounded Response

Embedding model: gemini-embedding-001

Embedding dimension: 768

9. Document and PPT Processing

Supported inputs:

- PDF
- DOCX
- PPT/PPTX
- Images / scanned documents

Processing technologies: PyMuPDF, python-docx, python-pptx, Pillow, Tesseract/pytesseract.

Scanned Document / Image
↓
Image Processing
↓
Tesseract OCR
↓
Extracted Text
↓
AI Analysis

10. Web Research and Source Traceability

The Web Research Agent uses Requests and BeautifulSoup to retrieve and parse online information for workflows requiring current external knowledge. Research can be passed into generation and associated with source information where supported.

User Query → Web Research Agent → Web Sources
→ Content Extraction → Relevant Information → Generation

11. Artifact Generation

DOCX

Generated using python-docx. Outputs are intended to remain editable and may contain headings, paragraphs, tables, sections, and references.

PPTX

Generated using python-pptx. Outputs are intended to remain editable and may contain titles, text, multiple slides, structured layouts, and template-based content.

The workflow is designed to use uploaded templates to preserve relevant tone, structure, formatting, and visual style where supported.

12. Conversational Editing and Versioning

User: "Add an executive summary."

↓

Editing Agent

↓

Updated Artifact

Original Artifact → Version 1 → User Edit → Version 2 → Additional Edit → Version 3

13. End-to-End Workflow

1. Upload Enterprise_Knowledge.pdf
2. Upload DOCX proposal template
3. Upload PPTX presentation template
4. Analyze templates
5. Perform web research
6. Retrieve enterprise knowledge
7. Generate content
8. Generate DOCX and PPTX
9. Validate artifacts
10. Review sources/citations
11. Perform conversational edits
12. Review versions

Example request: Research the latest Generative AI trends and create a proposal and a 12-slide presentation using the same tone and style as the uploaded files.

14. Technology Stack

Component	Technology
Language	Python
LLM	Google Gemini
Frontend	Streamlit
Backend	FastAPI
Vector Database	Pinecone
Embeddings	Gemini Embeddings
RAG	Retrieval-Augmented Generation
PDF	PyMuPDF
DOCX	python-docx
PPTX	python-pptx
Image	Pillow
OCR	Tesseract / pytesseract
Web Research	Requests + BeautifulSoup
Background Tasks	Celery
Message Broker	Redis
Validation	Pydantic

15. Project Structure

```
Multi-AI-Agent/  
├── app/  
│   ├── agents/  
│   ├── api/  
│   ├── document_processing/  
│   ├── generation/  
│   ├── models/  
│   ├── config.py  
│   └── main.py  
├── frontend/  
│   └── app.py  
├── samples/  
│   ├── Enterprise_Knowledge.pdf  
│   ├── Company_Proposal_Template.docx  
│   └── Company_Presentation_Template.pptx  
├── requirements.txt  
├── .env.example  
├── GEMINI_SETUP.md  
└── README.md
```

16. Configuration

```
GEMINI_API_KEY=your-gemini-api-key  
GEMINI_MODEL=gemini-2.5-flash  
EMBEDDING_MODEL=gemini-embedding-001  
EMBEDDING_DIMENSION=768
```

```
PINECONE_API_KEY=your-pinecone-api-key  
PINECONE_INDEX=multi-agent-rag-gemini  
PINECONE_NAMESPACE=default  
PINECONE_CLOUD=aws  
PINECONE_REGION=us-east-1
```

```
REDIS_URL=redis://localhost:6379/0  
DATA_DIR=./data  
UPLOAD_DIR=./uploads  
OUTPUT_DIR=./outputs  
WEB_MAX_RESULTS=5
```

17. Installation and Setup

1. Clone repository:

```
git clone https://github.com/shreyash-07-ai/Multi-AI-Agent.git  
cd Multi-AI-Agent
```

2. Create and activate virtual environment:

```
python -m venv .venv  
.venv\Scripts\activate
```

If PowerShell blocks activation:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Create .env:

```
copy .env.example .env
```

Add valid Gemini and Pinecone API keys to .env.

18. Running the Application

FastAPI Backend

```
uvicorn app.main:app --reload --port 8000
```

Backend: <http://localhost:8000>

Health check: <http://localhost:8000/health>

```
{"status": "ok", "service": "multi-agent-ai"}
```

Streamlit Frontend

```
streamlit run frontend/app.py
```

Normally available at <http://localhost:8501>.

Celery + Redis

Celery and Redis are included for background processing. The worker command must match the Celery application module present in the current project version. If no Celery application module exists, test the core synchronous workflow instead.

19. Testing and Sample Workflow

Recommended sample files:

- Enterprise_Knowledge.pdf
- Company_Proposal_Template.docx
- Company_Presentation_Template.pptx

RAG test questions:

- Where is NovaTech Solutions headquartered?
- What is the purpose of SecureRAG?
- What are the stages of the enterprise RAG pipeline?
- Can customer documents be used for model training without approval?
- What are NovaTech's strategic priorities for 2026?

Generation/editing tests:

- Generate a professional proposal using the uploaded DOCX template.
- Generate a 12-slide presentation using the uploaded PPTX template.

- Add an executive summary.
- Add two slides about market trends.
- Make the presentation more concise.

20. API Modules

Module	Purpose
app/api/upload.py	File upload and ingestion
app/api/chat.py	Chat requests and multi-agent workflow
app/api/artifacts.py	Generated artifact operations
app/api/versions.py	Version-related operations
/health	Backend health check

21. Troubleshooting

Backend connection refused

Start FastAPI with `uvicorn app.main:app --reload --port 8000` and keep it running.

Gemini API error

Check `GEMINI_API_KEY`, internet connectivity, and model availability.

Pinecone error

Check `PINECONE_API_KEY`, `PINECONE_INDEX`, and embedding dimension compatibility.

OCR error

Install Tesseract OCR itself; `pytesseract` alone is not sufficient.

Redis/Celery error

Ensure Redis is running and use the Celery application module that exists in the current project.

22. Security

- Keep API keys in `.env`.
- Do not commit `.env` to GitHub.
- Commit `.env.example` instead.
- Do not hard-code API keys in Python files.
- Add authentication and access controls before production deployment.

23. Assignment Requirement Coverage

Requirement	Project Component
Multi-agent chatbot	app/agents + FastAPI + Streamlit
Supervisor / Orchestrator	supervisor_agent.py
Document analysis	document_analysis_agent.py + processors
PPT analysis	ppt_analysis_agent.py
Real-time web research	web_research_agent.py
Enterprise RAG	rag_agent.py + Pinecone

PDF support	pdf_processor.py
DOCX support	docx_processor.py
PPT/PPTX support	pptx_processor.py
Image/scanned support	Image processing + OCR
OCR	ocr_processor.py / Tesseract
Editable DOCX	docx_generator.py
Editable PPTX	pptx_generator.py
Template/style preservation	Template analysis + generation workflow
Conversational editing	editing_agent.py
Document/PPT conversion	converter.py
Source/citation tracking	Research/source workflow
Validation	validation_agent.py
Version management	versions.py
Backend APIs	FastAPI
Frontend	Streamlit
Background processing	Celery + Redis
Sample templates	samples/
Documentation	README.md + project documentation
Dependencies	requirements.txt

24. Future Scope

- Cloud deployment and scalable infrastructure.
- Advanced authentication and role-based authorization.
- Improved template and visual-style preservation.
- Advanced agent memory and long-running conversations.
- Scalable asynchronous distributed processing.
- Improved document-to-PPT and PPT-to-document conversion.
- Enhanced citation validation and provenance tracking.
- Additional enterprise connectors and knowledge sources.

25. Conclusion

The Multi-Agent AI Document & PPT Generator demonstrates how specialized AI agents can collaborate to automate an enterprise document and presentation workflow. The system combines document processing, template analysis, web research, enterprise RAG, artifact generation, validation, conversational editing, and version-related capabilities in a modular architecture.